

# argoCD

## Tasks:

1. Setup ARGO CD
2. Create Jenkins job for CI.
3. Create Jenkins job to create docker image and build image.
4. Create ArgoCD job to deploy on ek8s cluster.

## Task:1

1. **Setup ARGO CD**

## STEP 1 — Install ArgoCD in Kubernetes

Login to **master node**:

```
kubectl create namespace argocd
```

```
[ubuntu@master:~$ kubectl create namespace argocd
namespace/argocd created
ubuntu@master:~$
```

```
[ubuntu@master:~$ kubectl get ns
NAME                STATUS    AGE
argocd               Active    3m16s
default              Active    13d
kube-node-lease      Active    13d
kube-public          Active    13d
kube-system          Active    13d
ubuntu@master:~$
```

## Install ArgoCD:

```
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

Wait:

```
kubectl get pods -n argocd
```

```
namespace/argocd created
ubuntu@master:~$ kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
customresourcedefinition.apiextensions.k8s.io/applications.argoproj.io created
customresourcedefinition.apiextensions.k8s.io/appprojects.argoproj.io created
serviceaccount/argocd-application-controller created
serviceaccount/argocd-applicationset-controller created
serviceaccount/argocd-dex-server created
serviceaccount/argocd-notifications-controller created
serviceaccount/argocd-redis created
serviceaccount/argocd-repo-server created
serviceaccount/argocd-server created
role.rbac.authorization.k8s.io/argocd-application-controller created
role.rbac.authorization.k8s.io/argocd-applicationset-controller created
role.rbac.authorization.k8s.io/argocd-dex-server created
role.rbac.authorization.k8s.io/argocd-notifications-controller created
role.rbac.authorization.k8s.io/argocd-redis created
role.rbac.authorization.k8s.io/argocd-server created
clusterrole.rbac.authorization.k8s.io/argocd-application-controller created
clusterrole.rbac.authorization.k8s.io/argocd-applicationset-controller created
clusterrole.rbac.authorization.k8s.io/argocd-server created
rolebinding.rbac.authorization.k8s.io/argocd-application-controller created
```

The CustomResourceDefinition "applicationsets.argoproj.io" is invalid: metadata.annotations:

```
ubuntu@master:~$ kubectl get pods -n argocd
```

| NAME                                              | READY | STATUS  | RESTARTS | AGE |
|---------------------------------------------------|-------|---------|----------|-----|
| argocd-application-controller-0                   | 1/1   | Running | 0        | 72s |
| argocd-applicationset-controller-7d4bfb5fdc-lpw4t | 1/1   | Running | 0        | 72s |
| argocd-dex-server-696fd7bfcb-hhpg4                | 1/1   | Running | 0        | 72s |
| argocd-notifications-controller-5d7cddc694-gvt5c  | 1/1   | Running | 0        | 72s |
| argocd-redis-5955475487-bv46c                     | 1/1   | Running | 0        | 72s |
| argocd-repo-server-5bd7df494d-vcgrt               | 1/1   | Running | 0        | 72s |
| argocd-server-78dc949c79-m9hbv                    | 1/1   | Running | 0        | 72s |

```
ubuntu@master:~$
```

## Step 3: Expose ArgoCD UI

By default it's ClusterIP (not accessible outside).

Change it to NodePort:

```
kubectl patch svc argocd-server -n argocd \
  -p '{"spec": {"type": "NodePort"}}'
```

Now check service:

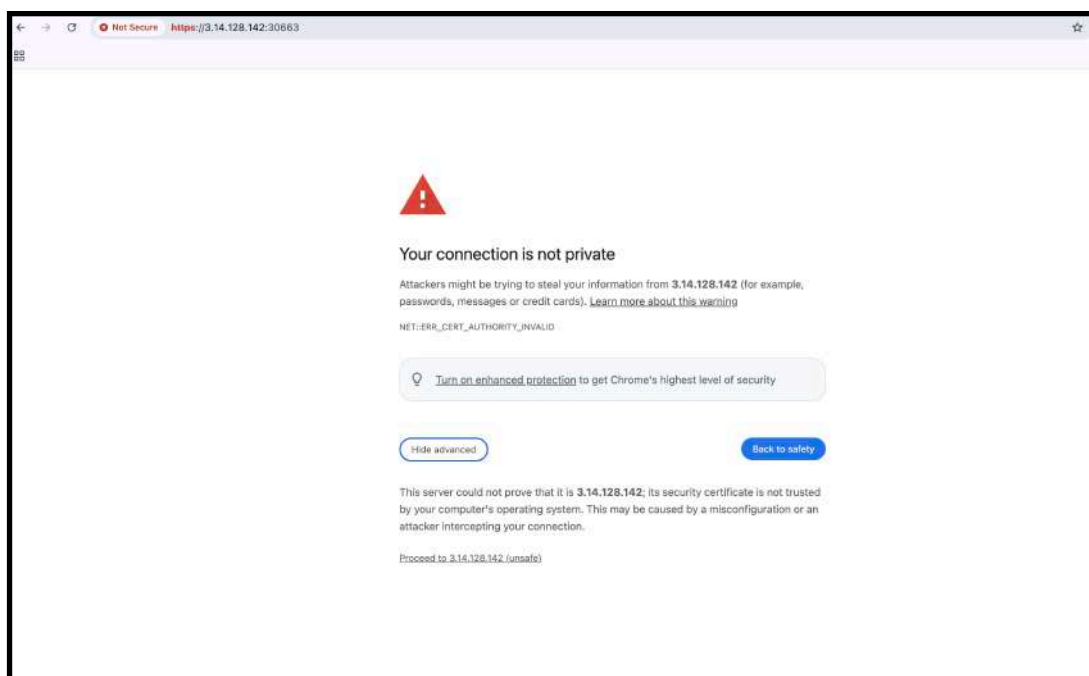
```
kubectl get svc -n argocd
```

```
ubuntu@master:~$ kubectl patch svc argocd-server -n argocd \
  -p '{"spec": {"type": "NodePort"}}'
service/argocd-server patched
ubuntu@master:~$ kubectl get svc -n argocd
```

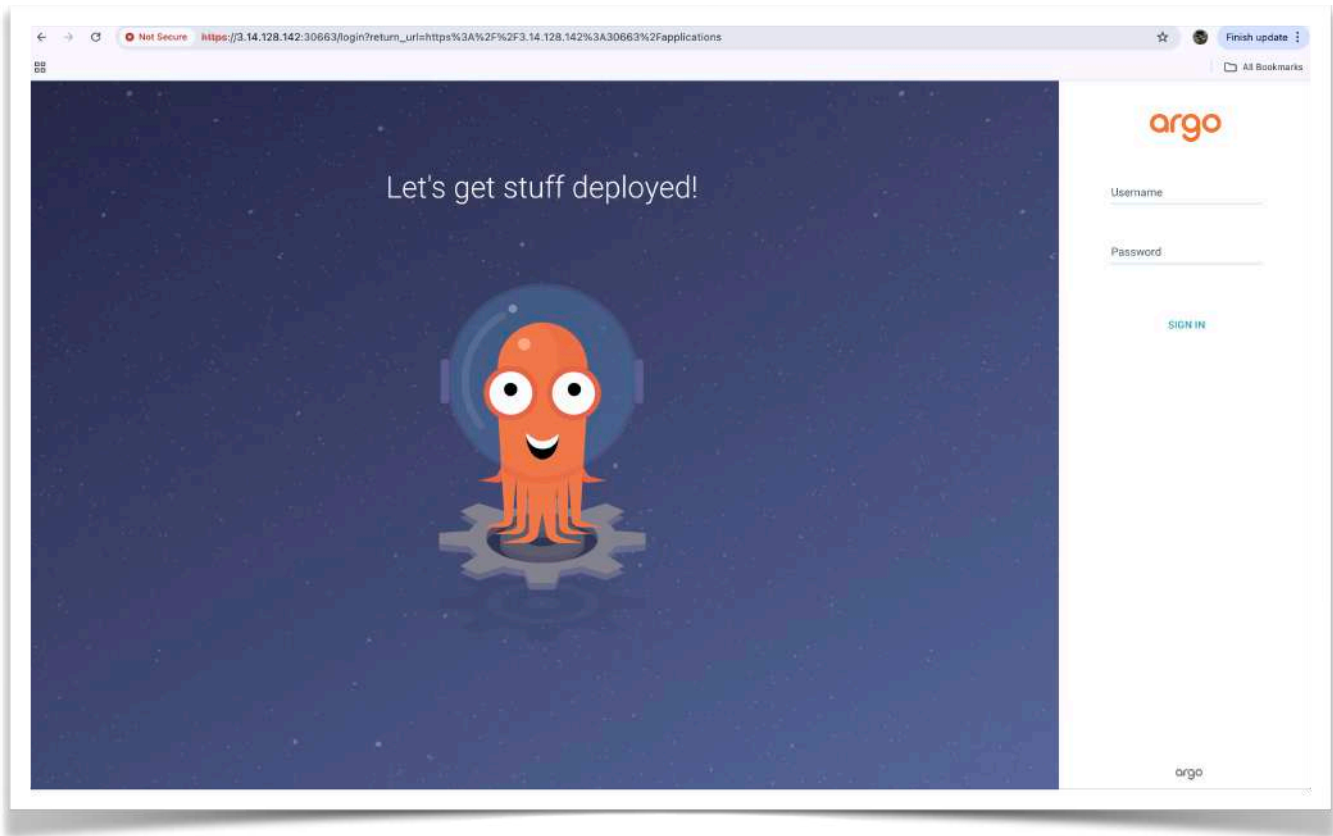
| NAME                                    | TYPE      | CLUSTER-IP     | EXTERNAL-IP | PORT(S)                      | AGE  |
|-----------------------------------------|-----------|----------------|-------------|------------------------------|------|
| argocd-applicationset-controller        | ClusterIP | 10.104.209.139 | <none>      | 7000/TCP, 8080/TCP           | 5m1s |
| argocd-dex-server                       | ClusterIP | 10.102.179.99  | <none>      | 5556/TCP, 5557/TCP, 5558/TCP | 5m1s |
| argocd-metrics                          | ClusterIP | 10.107.205.4   | <none>      | 8082/TCP                     | 5m1s |
| argocd-notifications-controller-metrics | ClusterIP | 10.97.142.158  | <none>      | 9001/TCP                     | 5m1s |
| argocd-redis                            | ClusterIP | 10.96.47.243   | <none>      | 6379/TCP                     | 5m1s |
| argocd-repo-server                      | ClusterIP | 10.103.46.138  | <none>      | 8081/TCP, 8084/TCP           | 5m   |
| argocd-server                           | NodePort  | 10.98.19.5     | <none>      | 80:30663/TCP, 443:30944/TCP  | 5m   |
| argocd-server-metrics                   | ClusterIP | 10.106.206.98  | <none>      | 8083/TCP                     | 5m   |

```
ubuntu@master:~$
```

—> now inside browser add the ip :30663



—> click the proceed



—> to get password below step

—Step 4: Get Admin Password

Run:

```
kubectl get secret argocd-initial-admin-secret -n argocd \
  -o jsonpath="{.data.password}" | base64 --decode
```

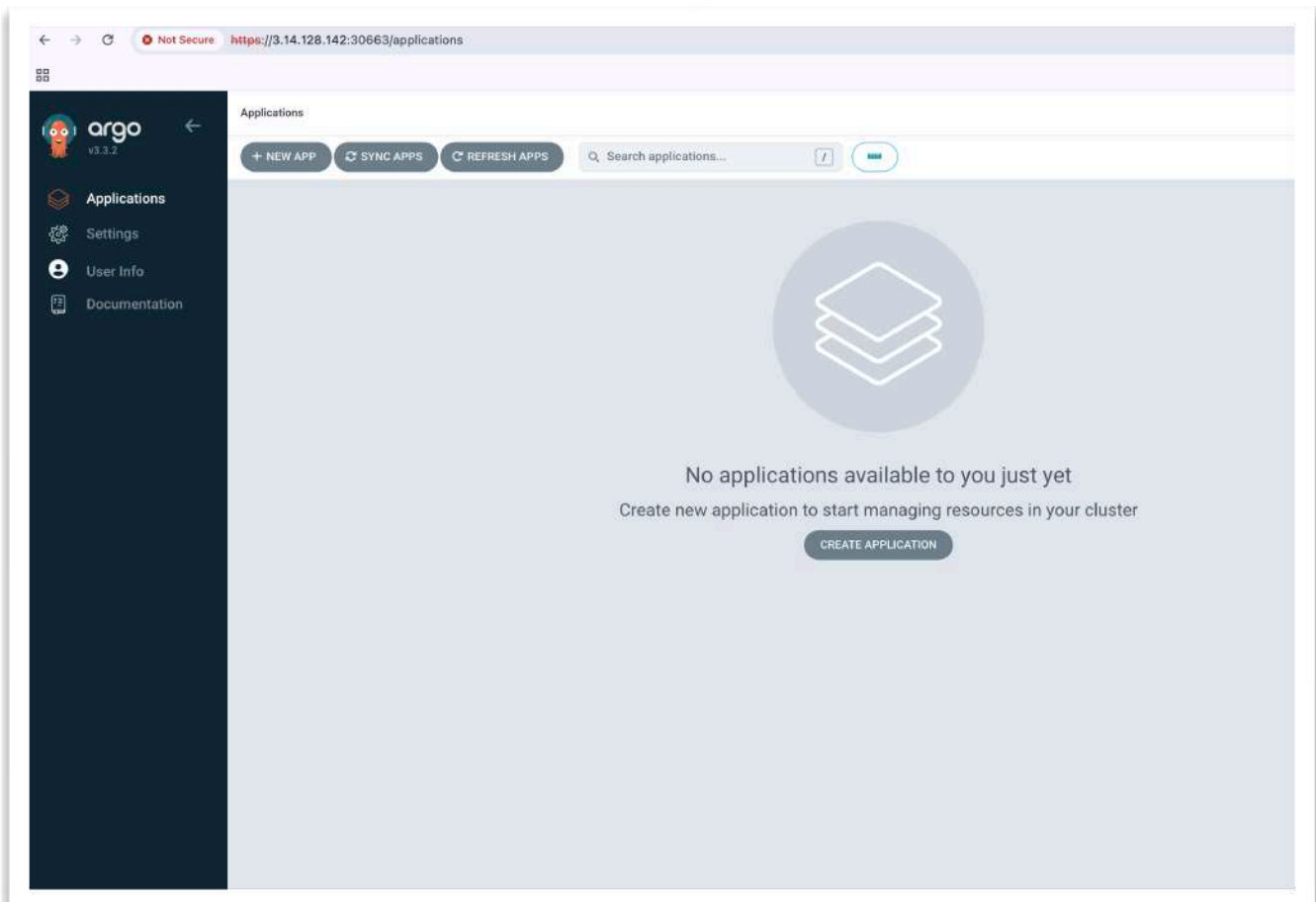
Username:

admin

Password:

(the output from above command)

```
argocd-server-metrics ClusterIP 10.106.206.98 <none>
ubuntu@master:~$ kubectl get secret argocd-initial-admin-secret -n argocd \
[ -o jsonpath="{.data.password}" | base64 --decode
XkH36Dc7SFoS0iCuubuntu@master:~$
```



## Task 1 Completed :

- ✓ Namespace created
- ✓ ArgoCD pods running
- ✓ UI accessible in browser
- ✓ Logged in successfully

## **Task:2 Create Jenkins job for CI.**

### **Jenkins CI Job**

- Checkout
- Maven build
- SonarQube analysis
- Deploy to Nexus

### **Tools Used in Jenkins Server**

1. Jenkins (Pipeline job)
2. Git (Source code management)
3. Maven (Build tool)
4. SonarQube (Code quality analysis)
5. Nexus (Artifact repository)

## **CI Flow Steps**

### **Configure Tools in Jenkins**

- Installed Maven in Jenkins (Global Tool Configuration)
- Configured SonarQube server in Jenkins
- Added Nexus credentials in Jenkins

### **Create Pipeline Job**

- Created new Pipeline job in Jenkins

- Connected job to GitHub repository
- Used Jenkinsfile stored in GitHub

## Checkout Stage

- Jenkins pulls source code from GitHub (main branch)

—> add this pipeline in Jenkins job pipeline

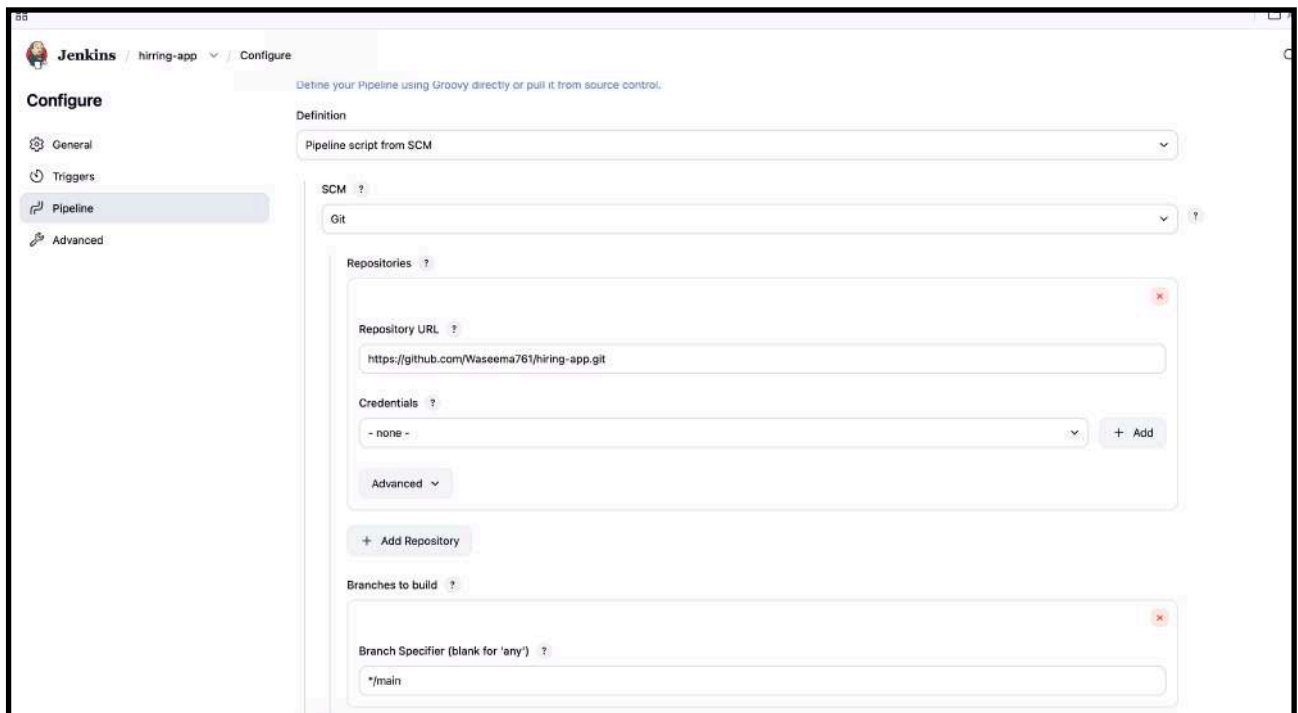


```
1 pipeline {
2   agent any
3
4   tools {
5     maven 'maven'
6   }
7
8   stages {
9
10    stage('Checkout') {
11      steps {
12        git branch: 'main',
13           url: 'https://github.com/Waseema761/hiring-app.git'
14      }
15    }
16
17    stage('Build') {
18      steps {
19        sh 'mvn clean package'
20      }
21    }
22
23    stage('SonarQube Analysis') {
24      steps {
25        withSonarQubeEnv('sonarqube') {
26          sh 'mvn sonar:sonar'
27        }
28      }
29    }
30
31    stage('Deploy to Nexus') {
32      steps {
33        sh 'mvn deploy'
34      }
35    }
36  }
37 }
```

—> open your Jenkins gui

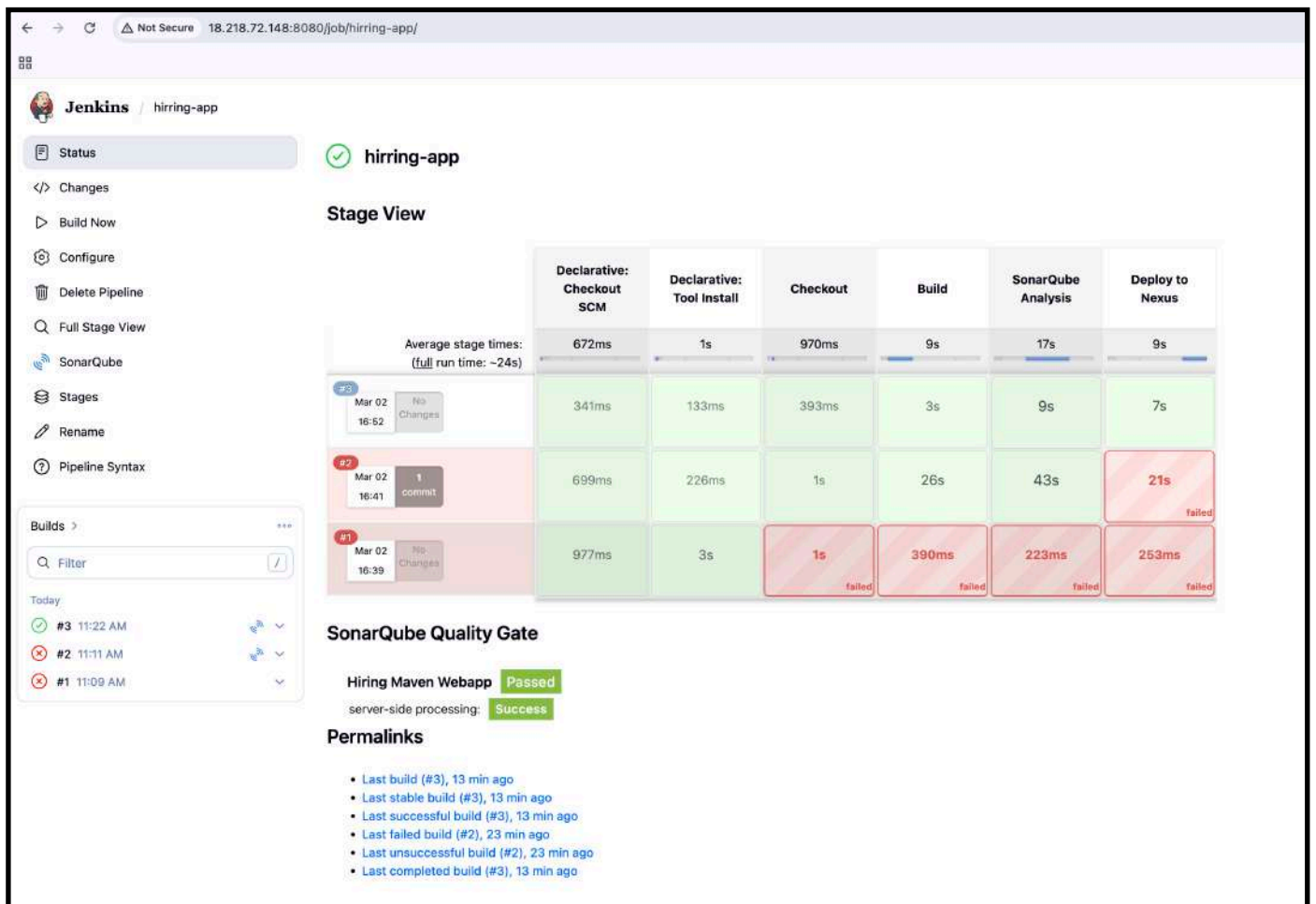
—> now create a job name herring-app

—> now add the scm git



—> build the job





## Result of CI

- ✓ Source code built successfully
- ✓ Code analyzed in SonarQube
- ✓ Artifact stored in Nexus
- ✓ Jenkins pipeline completed successfully

## Verify CI (Continuous Integration)

After creating Jenkins CI job, we verify using the following checks:

## Verify WAR File Created

Inside Jenkins workspace:

```
target/hiring.war
```

This confirms Maven build worked.

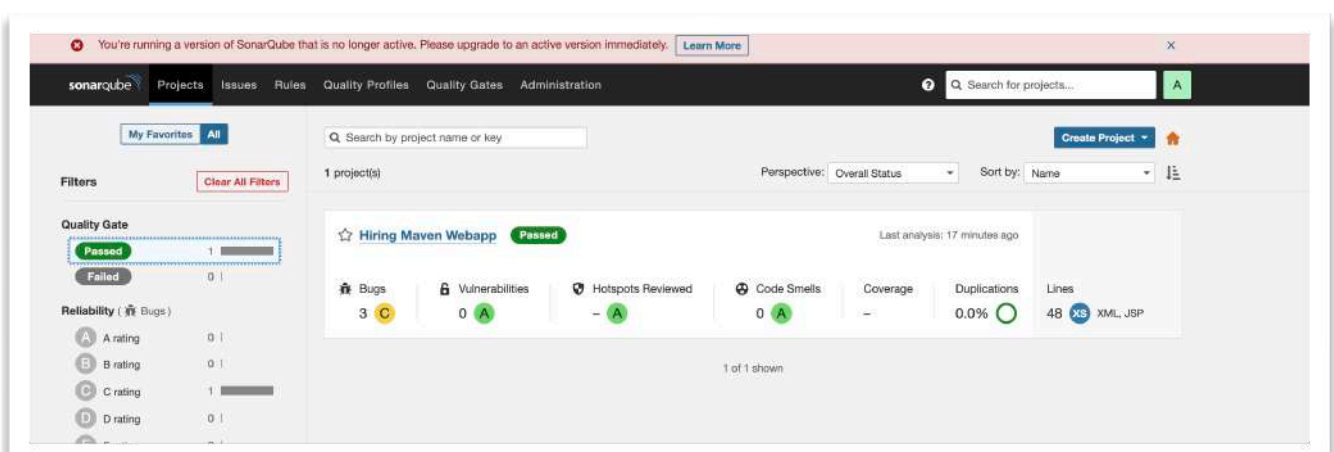
```
[root@ip-172-31-44-77 workspace]# pwd
/var/lib/jenkins/workspace
[root@ip-172-31-44-77 workspace]# ls
hiring-app  hiring-app@tmp
[root@ip-172-31-44-77 workspace]#
```

**i-0b60f82715f0f8ac8 (jenkins)**

PublicIPs: 18.218.72.148 PrivateIPs: 172.31.44.77

## Verify SonarQube Analysis

- Login to SonarQube UI
- Open project (Hiring Maven Webapp)
- Check:
  - Quality Gate → Passed
  - No major critical issues

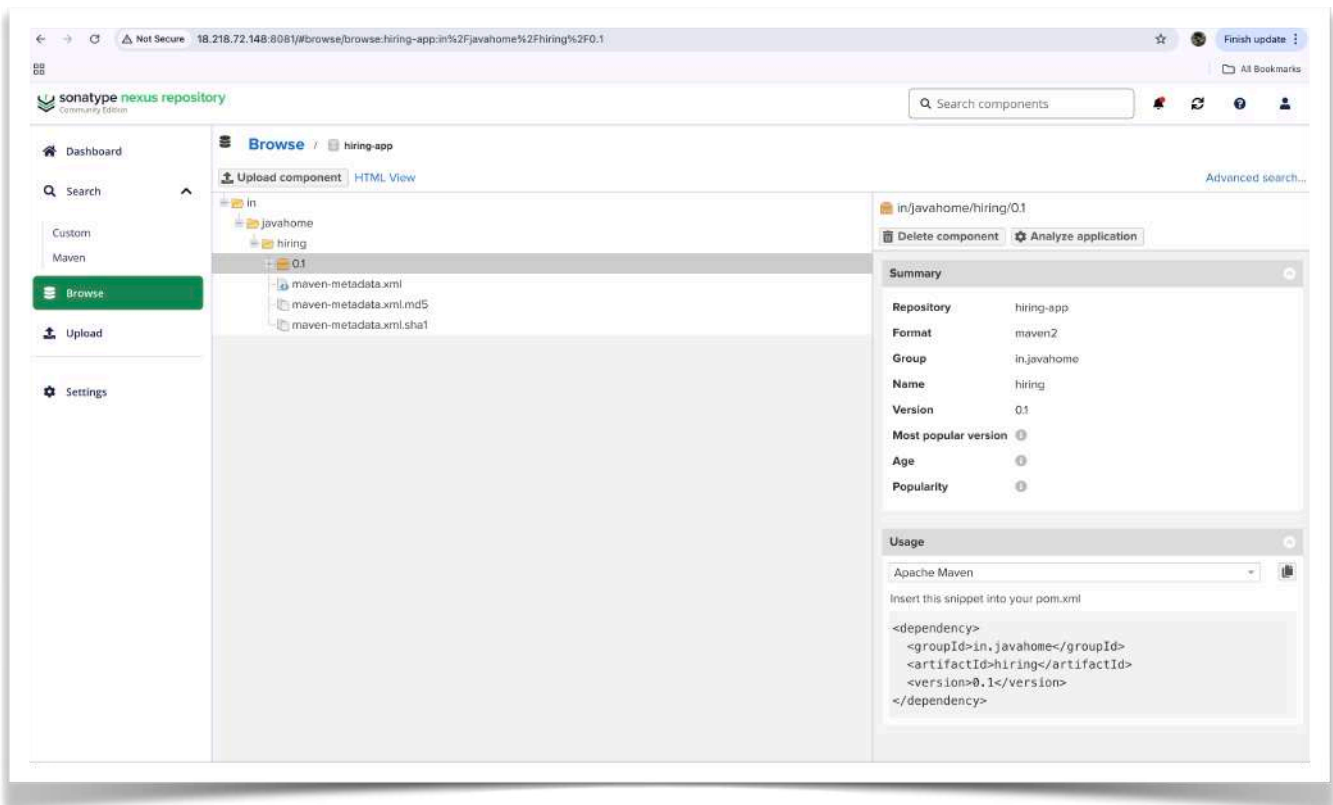


## Verify Artifact in Nexus

- Login to Nexus
- Go to Browse → hiring-app repository
- Check path:

in/javahome/hiring/0.1/

- Confirm files:
  - hiring-0.1.war
  - hiring-0.1.pom



## Final CI Verification Summary

- ✓ Jenkins build successful
- ✓ WAR artifact generated
- ✓ SonarQube Quality Gate passed
- ✓ Artifact uploaded to Nexus

That is proper **CI verification section**

## Task 3

### Create Jenkins job to create Docker image and build image

#### Objective

To containerize the application by building a Docker image using Jenkins pipeline and pushing it to DockerHub.

### Tools Used in Task 3 Pipeline

1. GitHub – Source code
2. Jenkins – CI/CD automation
3. Maven – Build tool
4. SonarQube – Code quality analysis
5. Nexus – Artifact storage (WAR file)
6. Docker – Image creation
7. DockerHub – Image registry

### Implementation Steps

#### Step 1: Ensure CI Pipeline Is Working

Before Docker stages, confirm:

1. Source code checkout from GitHub
2. Maven build generates WAR file
3. SonarQube analysis completes successfully
4. WAR artifact is uploaded to Nexus

Command used in CI stage:

## Step 2: Install Docker on Jenkins Server

On Jenkins server:

1. Install Docker
2. Start Docker service
3. Enable Docker on boot
4. Add Jenkins user to Docker group
5. Restart Jenkins

Verify installation:

```
docker --version
```

```
[root@ip-172-31-44-77 conf]# vi sonar-scanner.properties
[root@ip-172-31-44-77 conf]# docker --version
Docker version 25.0.14, build 0bab007
[root@ip-172-31-44-77 conf]#
```

**i-0b60f82715f0f8ac8 (jenkins)**

PublicIPs: 3.21.170.95 PrivateIPs: 172.31.44.77

## Step 3: Create DockerHub Credentials in Jenkins

1. Go to Manage Jenkins
2. Open Credentials
3. Add new credential
4. Type: Username and Password
5. Provide DockerHub username and password (or access token)
6. Set Credential ID as:

docker-hub



## Step 4: Create Dockerfile in Repository

Dockerfile stored in GitHub repository:

 main hiring-app / Dockerfile 

 **Waseema761** Update Dockerfile

Code

Blame

16 lines (11 loc) · 371 Bytes

```
1 FROM tomcat:8.5-jdk8-temurin
2
3 # Install curl
4 RUN apt-get update && apt-get install -y curl
5
6 # Nexus details
7 ENV NEXUS_URL=http://3.21.170.95:8081
8 ENV ARTIFACT_PATH=/repository/hiring-app/in/javahome/hiring/0.1/hiring-0.1.war
9
10 # Download WAR from Nexus
11 RUN curl -o /usr/local/tomcat/webapps/ROOT.war \
12     $NEXUS_URL$ARTIFACT_PATH
13
14 EXPOSE 8080
15
16 CMD ["catalina.sh", "run"]
```

Purpose:

- Uses Tomcat base image
- Installs curl
- Downloads WAR file from Nexus during build
- Deploys WAR as ROOT application
- Exposes port 8080

# Step:5

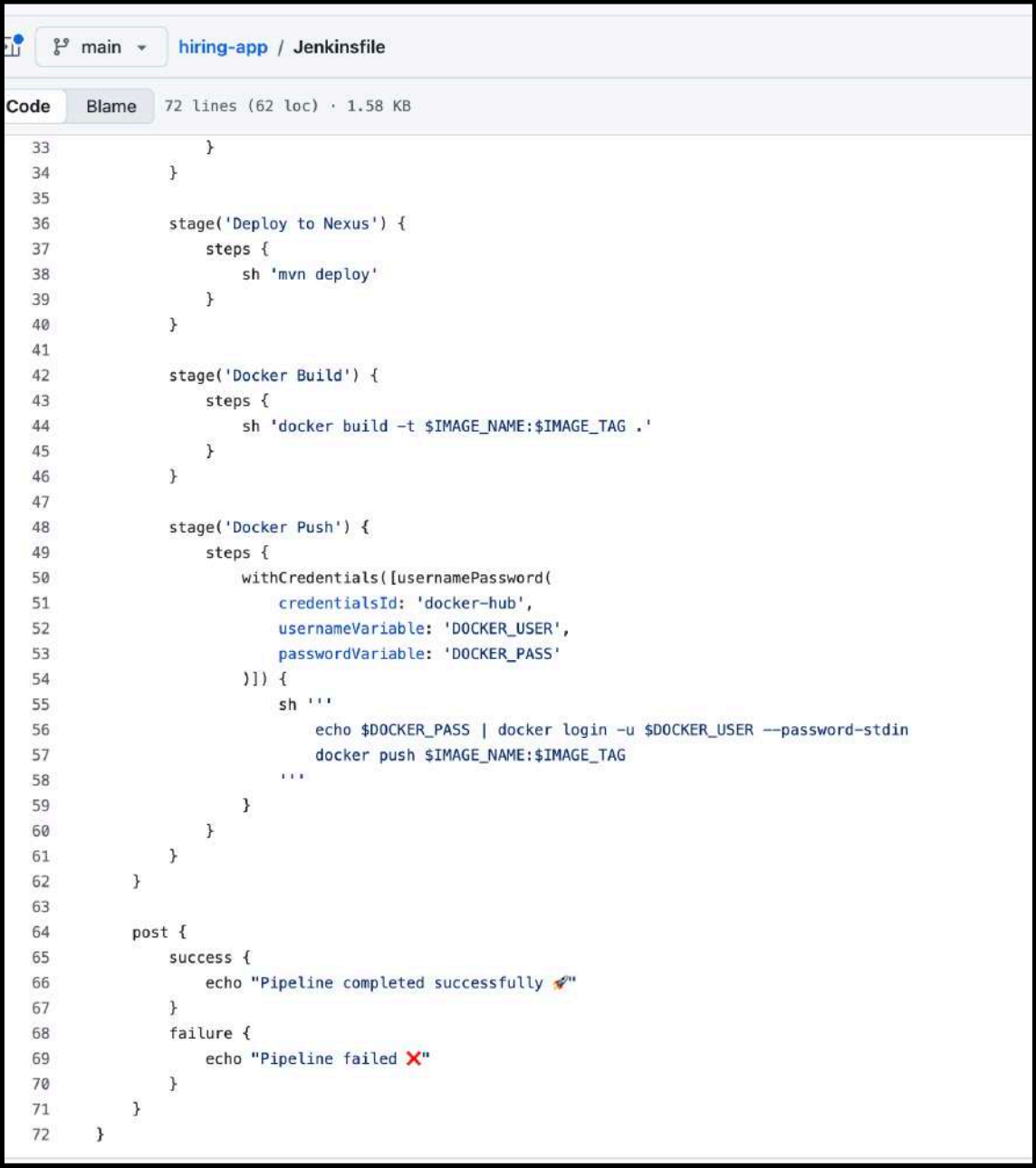
## Update Jenkins Pipeline

Docker stages added to existing CI pipeline.

Stages added:

- Docker Build
- Docker Push

Docker Build stage:



```
33     }
34 }
35
36 stage('Deploy to Nexus') {
37     steps {
38         sh 'mvn deploy'
39     }
40 }
41
42 stage('Docker Build') {
43     steps {
44         sh 'docker build -t $IMAGE_NAME:$IMAGE_TAG .'
45     }
46 }
47
48 stage('Docker Push') {
49     steps {
50         withCredentials([usernamePassword(
51             credentialsId: 'docker-hub',
52             usernameVariable: 'DOCKER_USER',
53             passwordVariable: 'DOCKER_PASS'
54         )]) {
55             sh '''
56                 echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin
57                 docker push $IMAGE_NAME:$IMAGE_TAG
58             '''
59         }
60     }
61 }
62 }
63
64 post {
65     success {
66         echo "Pipeline completed successfully 🎉"
67     }
68     failure {
69         echo "Pipeline failed ❌"
70     }
71 }
72 }
```



Docker Push stage:

- Login using Jenkins credentials
- Push image to DockerHub

## Complete Pipeline Flow

The complete flow executed in Task 3:

1. Checkout source code from GitHub
2. Build application using Maven
3. Perform SonarQube analysis
4. Deploy WAR artifact to Nexus
5. Build Docker image
6. Push Docker image to DockerHub

—> now inside the Jenkins job

Add the repo scm git

—-> and build the job



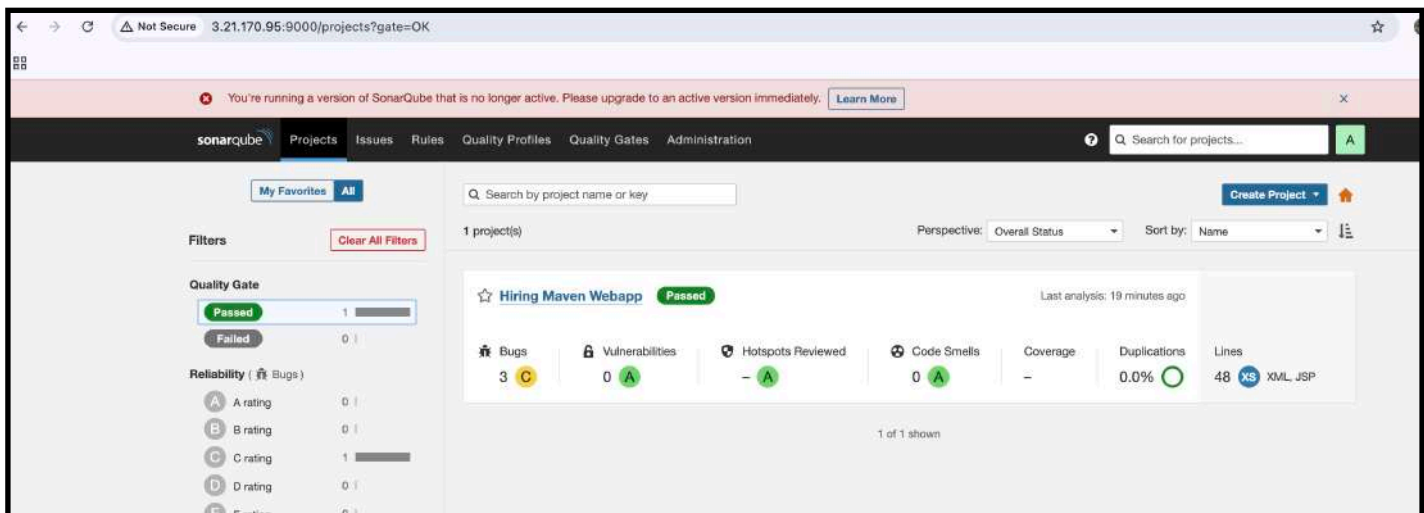
# Verification Steps

## —> 1 Verify Jenkins Pipeline

- Open Jenkins UI
- Check latest build status
- Confirm status is SUCCESS
- Confirm Docker Build and Docker Push stages passed

## 2 Verify SonarQube

- Open SonarQube UI
- Confirm project exists
- Confirm latest analysis is successful
- Confirm Quality Gate status

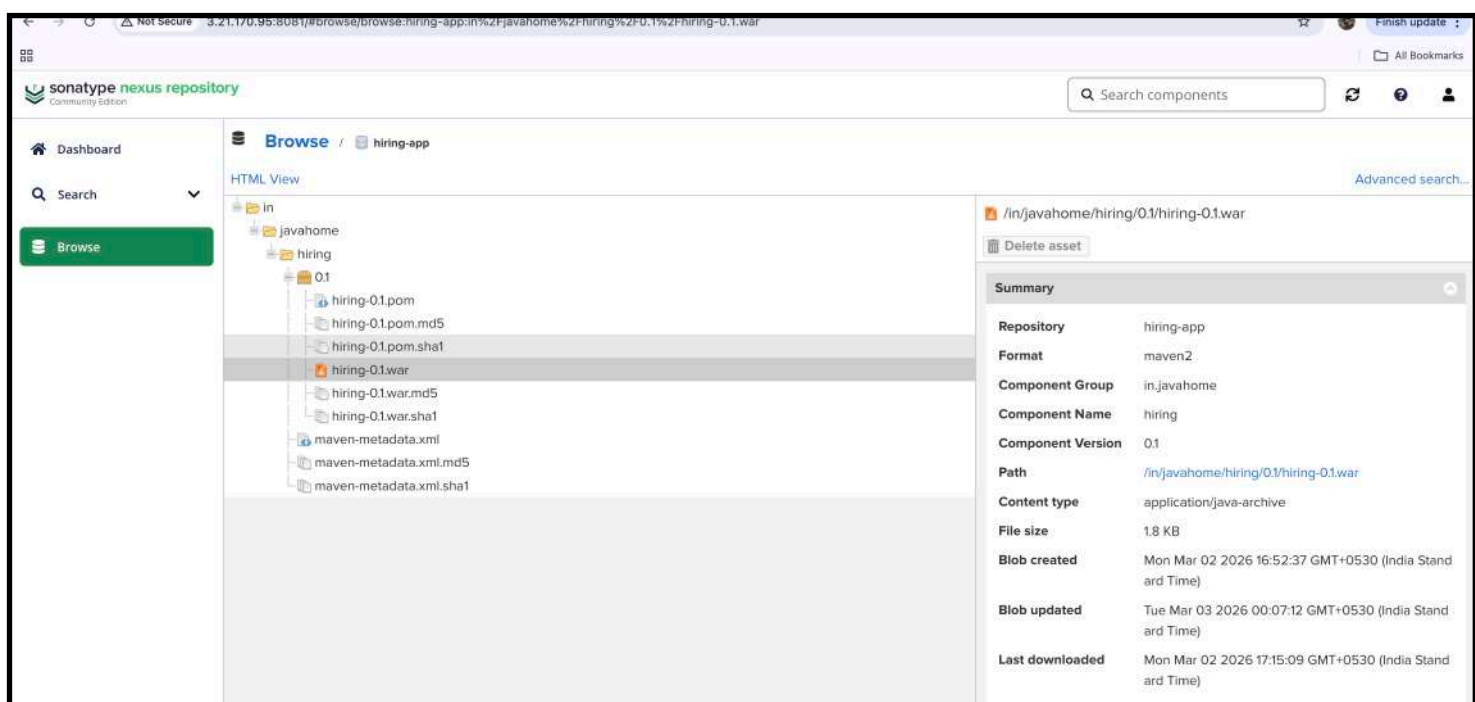


### 3 Verify Nexus

- Open Nexus UI
- Navigate to hiring-app repository
- Confirm artifact path:

in/javahome/hiring/0.1/

•



### 4 Verify Docker Image Locally

On Jenkins server:

docker images

Confirm image exists with correct tag.

```

docker version 23.0.14, build 0bab007
[root@ip-172-31-44-77 conf]# docker images
REPOSITORY          TAG             IMAGE ID        CREATED         SIZE
waseem08/hiring-app 9               7cfc912dc82a   23 minutes ago 414MB
waseema761/hiring-app 8             7cfc912dc82a   23 minutes ago 414MB
waseema761/hiring-app 4             292ecab9e1f7   7 hours ago    414MB
sonatype/nexus3      latest         9d8b132c1077   2 weeks ago    701MB
sonargube             lts           b2698b16c2b2   6 months ago   604MB
[root@ip-172-31-44-77 conf]#

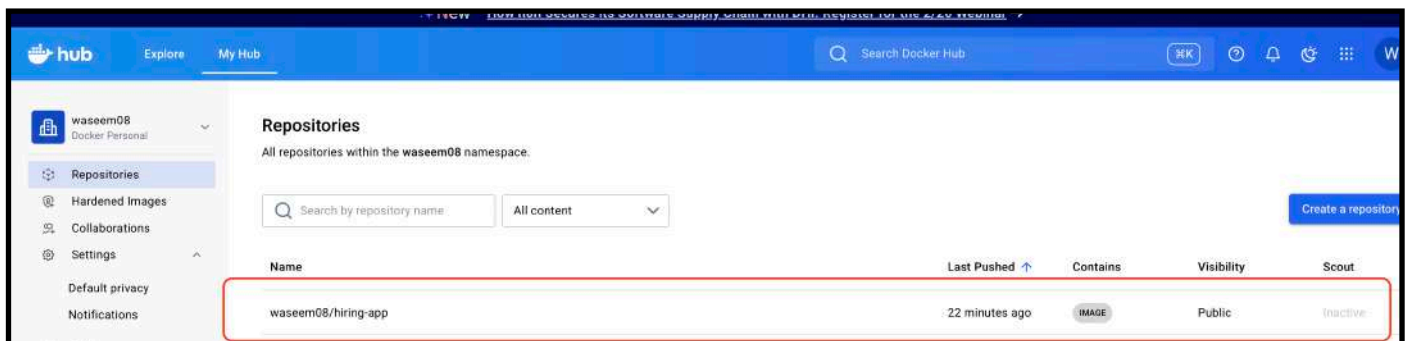
```

i-0b60f82715f0f8ac8 (jenkins)

PublicIPs: 3.21.170.95 PrivateIPs: 172.31.44.77

## 5 Verify DockerHub

- Login to DockerHub
- Open repository hiring-app
- Confirm new tag exists
- Confirm timestamp matches Jenkins build time



## 6 Optional: Run Container for Testing

### Run the Docker Container

Since Jenkins is using port 8080, use a different host port such as 9090.

```

docker run -d -p 9090:8080 --name hiring-container waseem08/hiring-app:9

```

Explanation:

- `-d` → Run in detached mode
- `-p 9090:8080` → Host port 9090 mapped to container port 8080
- `--name` → Assign container name
- `waseem08/hiring-app:9` → Image with tag

```
root@ip-172-31-44-77 conf]# docker run -d -p 9090:8080 --name hiring-container waseem08/hiring-app:9
356ef9cc776d70f6df33992ddbc0f18fa04ae6c2e1f368896ad4f1594b58e93c
root@ip-172-31-44-77 conf]#
```

i-0b60f82715f0f8ac8 (jenkins)

PublicIPs: 3.21.170.95 PrivateIPs: 172.31.44.77

Open in browser:

`http://<server-ip>:9090`

If application loads, image is working correctly.



# Result

- Application successfully built
- Code quality validated
- WAR artifact stored in Nexus
- Docker image created successfully
- Docker image pushed to DockerHub
- Image ready for Kubernetes deployment

## Task 4

### Create ArgoCD Application to Deploy Docker Image on Kubernetes Cluster

#### 1. Objective

To deploy the Docker image (`waseem08/hiring-app`) into the Kubernetes cluster using ArgoCD and implement GitOps-based deployment.

#### 2. Tools Used

- Kubernetes (EKS cluster)
- ArgoCD
- GitHub (for Kubernetes manifests)
- DockerHub (image registry)
- kubectl

### 3. Prerequisites

Before starting Task 4, ensure:

1. Kubernetes cluster is running

```
kubectl get nodes
```

```
argocd-server-metrics ClusterIP 10.106.206.98
[ubuntu@master:~]$ kubectl get nodes
NAME          STATUS    ROLES          AGE    VERSION
master        Ready     control-plane   14d    v1.31.14
worker1       Ready     <none>          14d    v1.31.14
worker2       Ready     <none>          14d    v1.31.14
ubuntu@master:~$
```

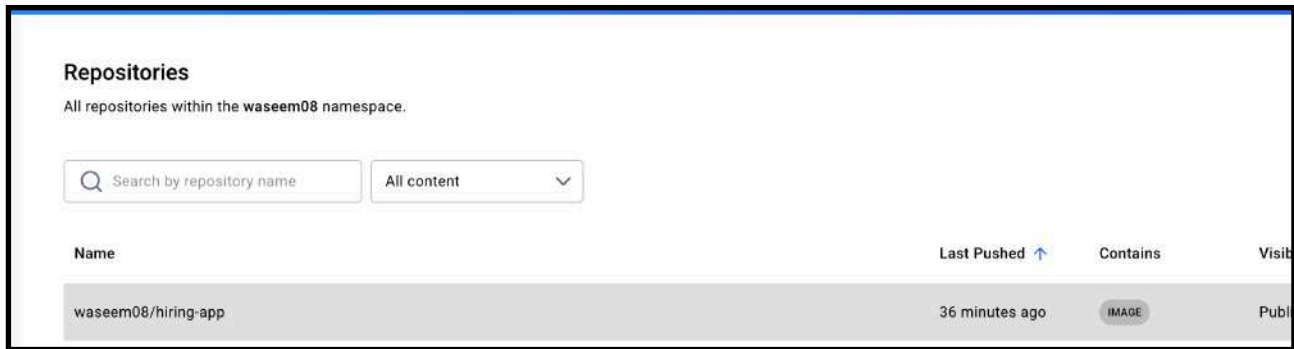
2. ArgoCD is installed in cluster

```
kubectl get pods -n argocd
```

```
kube-system Active 14d
[ubuntu@master:~]$ kubectl get svc -n argocd
NAME                                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)                                AGE
argocd-applicationset-controller    ClusterIP     10.104.209.139 <none>         7000/TCP,8080/TCP                      8h
argocd-dex-server                   ClusterIP     10.102.179.99  <none>         5556/TCP,5557/TCP,5558/TCP            8h
argocd-metrics                       ClusterIP     10.107.205.4   <none>         8082/TCP                               8h
argocd-notifications-controller-metrics ClusterIP     10.97.142.158  <none>         9001/TCP                               8h
argocd-redis                         ClusterIP     10.96.47.243   <none>         6379/TCP                               8h
argocd-repo-server                  ClusterIP     10.103.46.138  <none>         8081/TCP,8084/TCP                     8h
argocd-server                       NodePort      10.98.19.5     <none>         80:30663/TCP,443:30944/TCP            8h
argocd-server-metrics               ClusterIP     10.106.206.98  <none>         8083/TCP                               8h
[ubuntu@master:~]$ kubectl get nodes
```

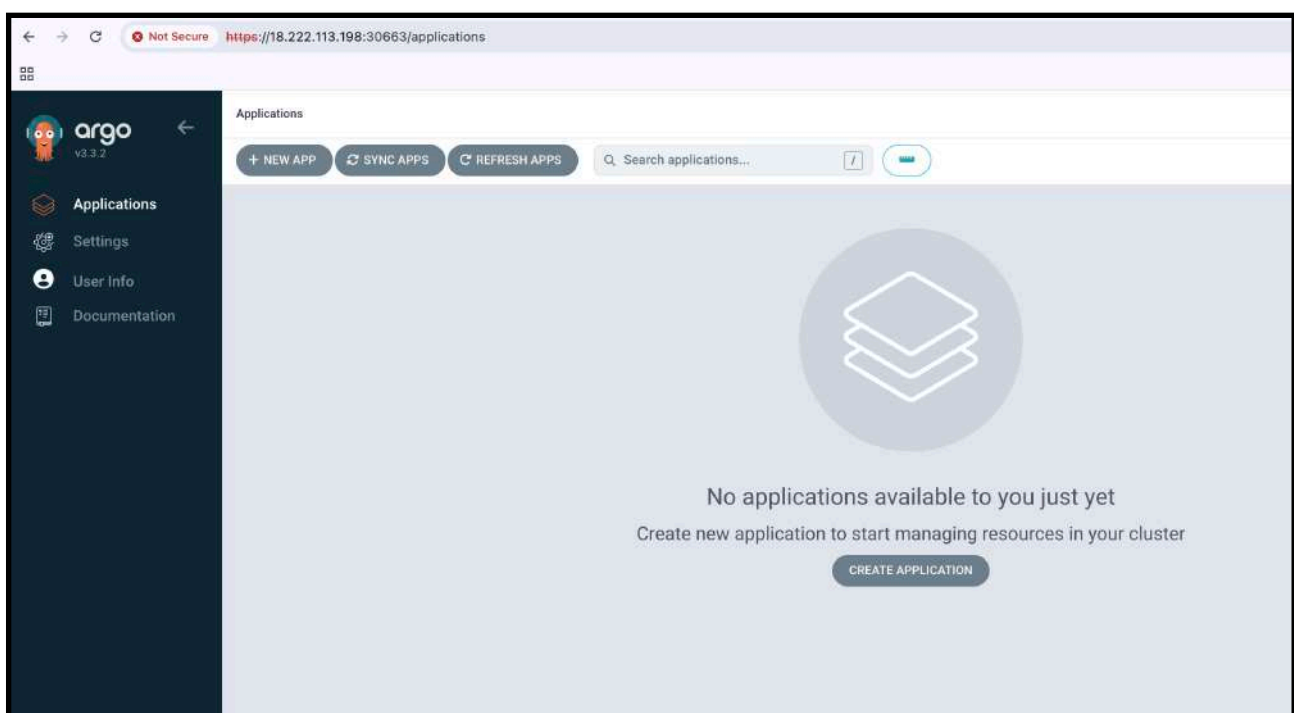
3 Docker image exists in DockerHub

waseem08/hiring-app:<tag>



4

—> argocd running





## Step:1

### Prepare GitHub Repository for Kubernetes Manifests

Waseema761/Hiring-app-argocd

Structure must be:

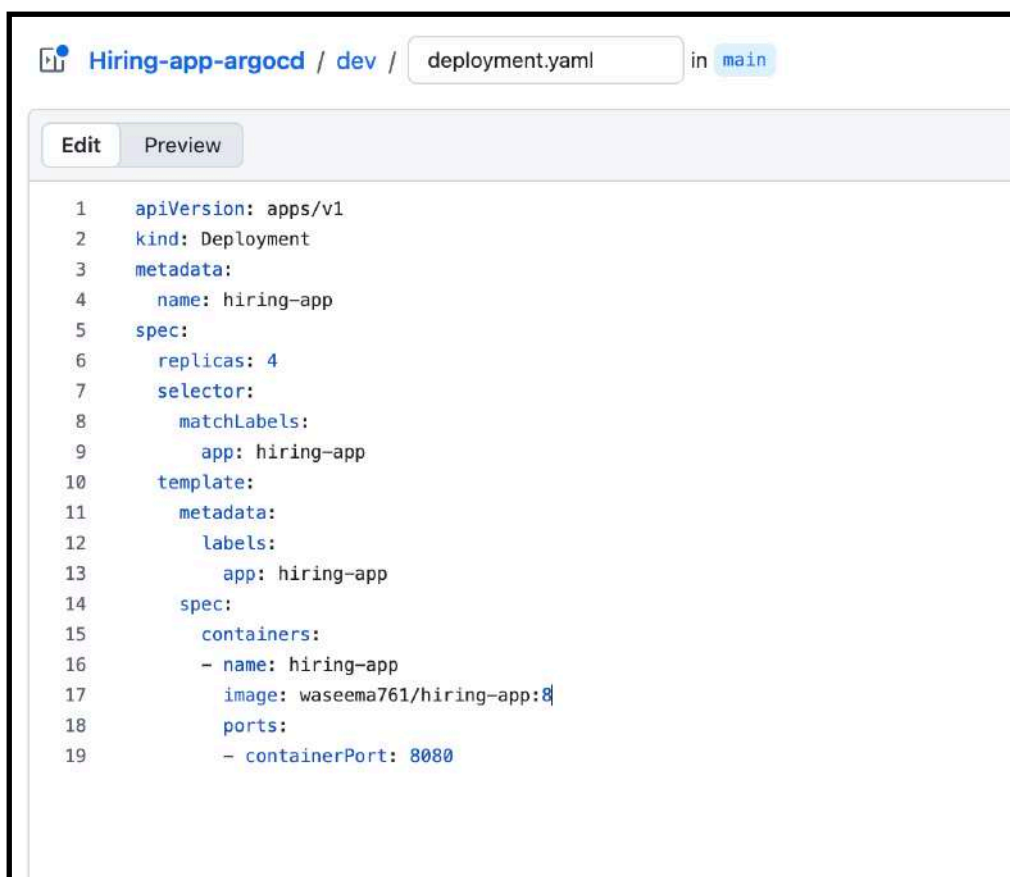
```
Hiring-app-argocd
├── application.yaml
└── dev/
    ├── deployment.yaml
    └── service.yaml
```

## STEP 2 – Update deployment.yaml

Inside dev/deployment.yaml:

- Use your Docker image
- Set replicas to 4

Example: my docker hub username/repo is waseem08/hiring-app

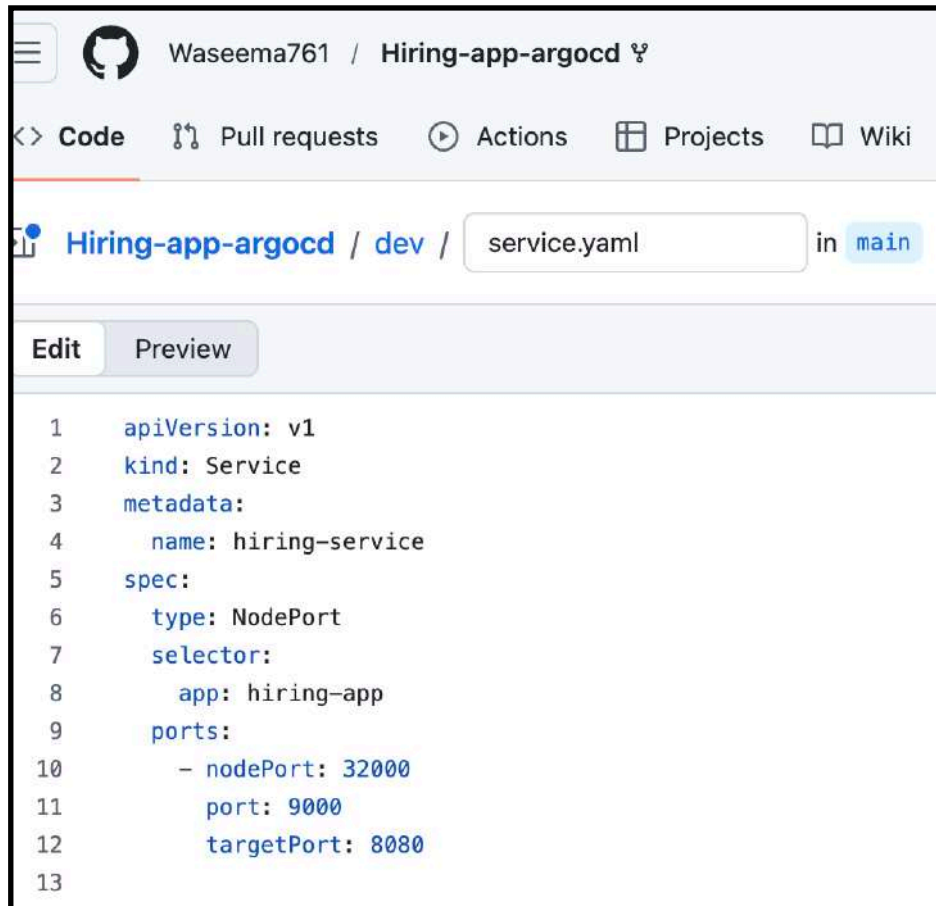


The screenshot shows a GitHub repository named 'Hiring-app-argocd' with the 'dev' branch selected. The file 'deployment.yaml' is open for editing. The code is as follows:

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: hiring-app
5  spec:
6    replicas: 4
7    selector:
8      matchLabels:
9        app: hiring-app
10   template:
11     metadata:
12       labels:
13         app: hiring-app
14     spec:
15       containers:
16       - name: hiring-app
17         image: waseema761/hiring-app:8
18         ports:
19         - containerPort: 8080
```

## STEP 3 – Verify service.yaml

Inside dev/service.yaml:



The screenshot shows a GitHub repository named 'Hiring-app-argocd' by user 'Waseema761'. The 'Code' tab is selected, showing the 'dev' branch with the file 'service.yaml'. The file content is as follows:

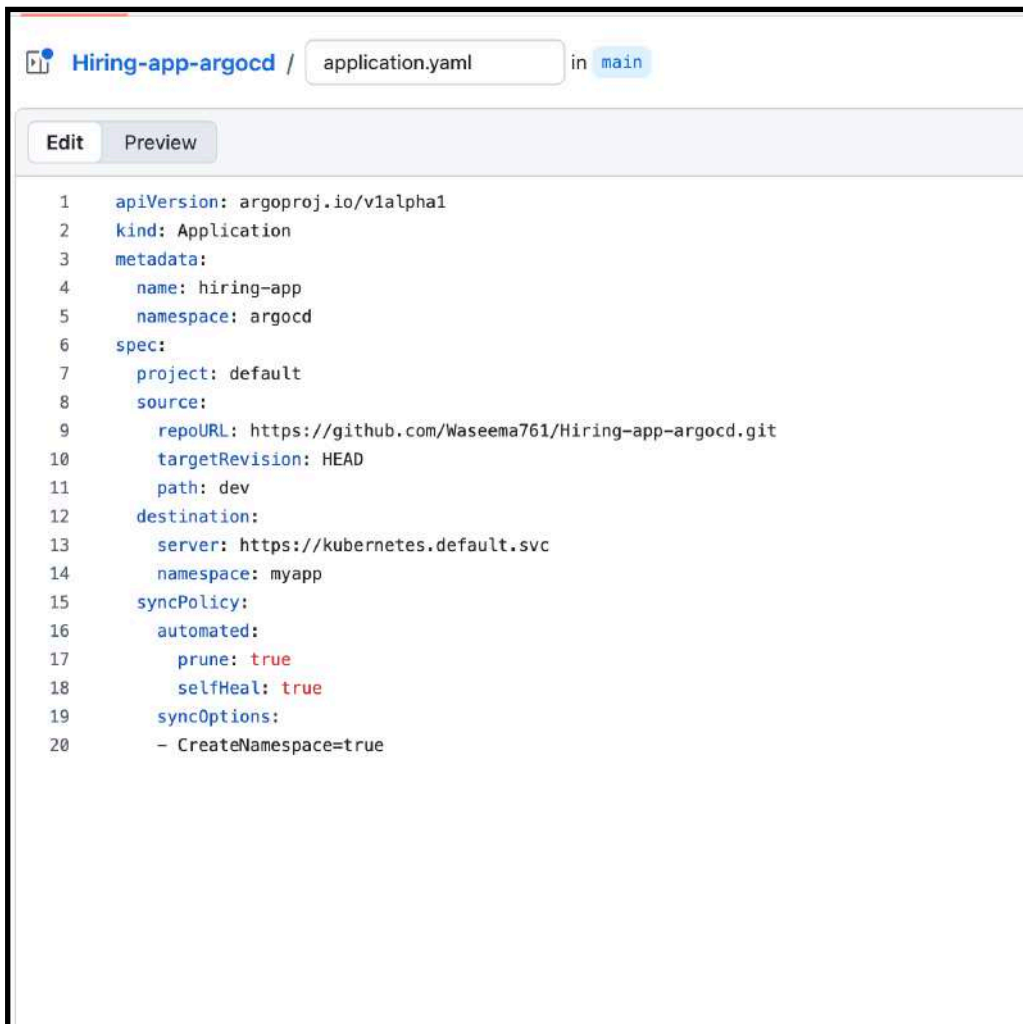
```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: hiring-service
5  spec:
6    type: NodePort
7    selector:
8      app: hiring-app
9    ports:
10     - nodePort: 32000
11       port: 9000
12       targetPort: 8080
13
```

## STEP 4 – Configure application.yaml (This Creates the ArgoCD Job)

application.yaml must contain:

- Your repo URL
- Path: dev
- Namespace: myapp
- Automated sync enabled

Example:



The screenshot shows the ArgoCD web interface. At the top, there's a header with the logo and the text 'Hiring-app-argocd / application.yaml in main'. Below this, there are two tabs: 'Edit' and 'Preview'. The 'Edit' tab is active, displaying the content of the application.yaml file. The file is a YAML document defining an ArgoCD Application resource. It includes fields for apiVersion, kind, metadata (name, namespace), spec (project, source, destination, syncPolicy), and syncOptions. The source is a Git repository, and the destination is a Kubernetes cluster. The syncPolicy is set to automated with prune and selfHeal enabled. The syncOptions include CreateNamespace=true.

```
1  apiVersion: argoproj.io/v1alpha1
2  kind: Application
3  metadata:
4    name: hiring-app
5    namespace: argocd
6  spec:
7    project: default
8    source:
9      repoURL: https://github.com/Waseema761/Hiring-app-argocd.git
10     targetRevision: HEAD
11     path: dev
12   destination:
13     server: https://kubernetes.default.svc
14     namespace: myapp
15   syncPolicy:
16     automated:
17       prune: true
18       selfHeal: true
19   syncOptions:
20     - CreateNamespace=true
```

## STEP 5 – Apply ArgoCD Application

Now go to master node.

You can either clone once:

```
git clone https://github.com/Waseema761/Hiring-app-argocd.git
cd Hiring-app-argocd
kubectl apply -f application.yaml
```

```
ubuntu@master:~$ git clone https://github.com/Waseema761/Hiring-app-argocd.git
Cloning into 'Hiring-app-argocd'...
remote: Enumerating objects: 180, done.
remote: Counting objects: 100% (50/50), done.
remote: Compressing objects: 100% (24/24), done.
remote: Total 180 (delta 40), reused 26 (delta 26), pack-reused 130 (from 1)
Receiving objects: 100% (180/180), 23.01 KiB | 7.67 MiB/s, done.
Resolving deltas: 100% (70/70), done.
ubuntu@master:~$ cd Hiring-app-argocd/
ubuntu@master:~/Hiring-app-argocd$ ls
README.md  application.yaml  dev
ubuntu@master:~/Hiring-app-argocd$
```

## STEP 6 – Verify ArgoCD Application

```
kubectl get applications -n argocd
```

You should see:

Application

```
ubuntu@master:~/Hiring-app-argocd/dev$ kubectl get applications -n argocd
NAME                                SYNC STATUS  HEALTH STATUS
myapp-argo-application             Synced       Healthy
ubuntu@master:~/Hiring-app-argocd/dev$
```

## Step 7 – Verify Deployment

```
kubectl get pods -n myapp
```

Expected:

4 pods

Running 1/1

```
ubuntu@master:~/Hiring-app-argocd/dev$ vi deployment.yaml
ubuntu@master:~/Hiring-app-argocd/dev$ kubectl get pods -n myapp
NAME                                READY    STATUS    RESTARTS   AGE
hiring-app-6b998d666f-gp8cj         1/1     Running   0           49s
hiring-app-6b998d666f-gr48w         1/1     Running   0           49s
hiring-app-6b998d666f-k2txl         1/1     Running   0           55s
hiring-app-6b998d666f-n8d86         1/1     Running   0           55s
```

## Step 8 – Verify Service

```
kubectl get svc -n myapp
```

Expected:

NodePort 32000

```
ubuntu@master:~/Hiring-app-argocd/dev$ kubectl get applications -n argocd
NAME                                SYNC STATUS  HEALTH STATUS
myapp-argo-application              Synced       Healthy
ubuntu@master:~/Hiring-app-argocd/dev$ kubectl get svc -n myapp
NAME            TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
hiring-service  NodePort    10.109.179.136 <none>         9000:32000/TCP   13m
ubuntu@master:~/Hiring-app-argocd/dev$
```

## Step 9 – Access Application

Open in browser:

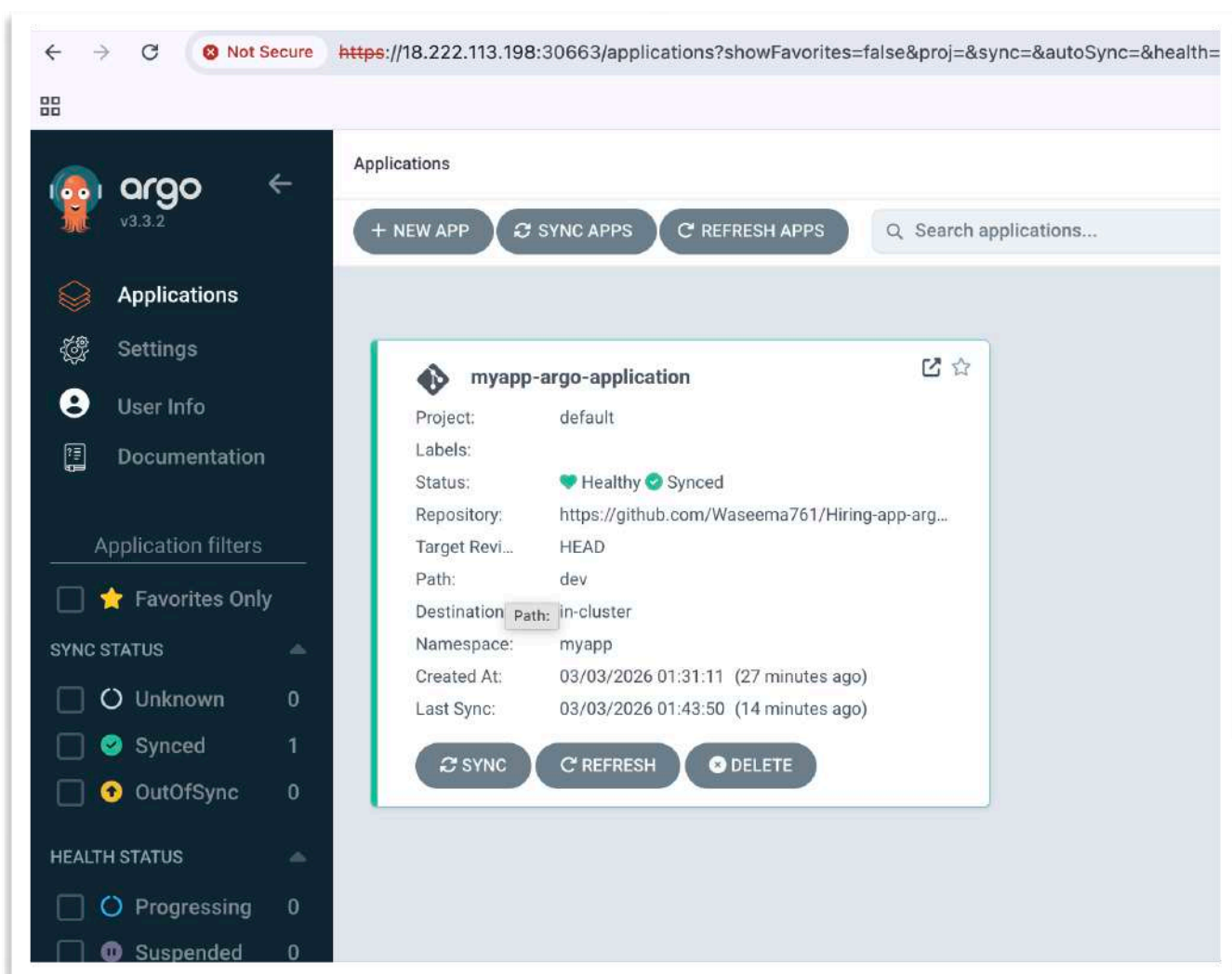
<http://<worker-node-public-ip>:32000>

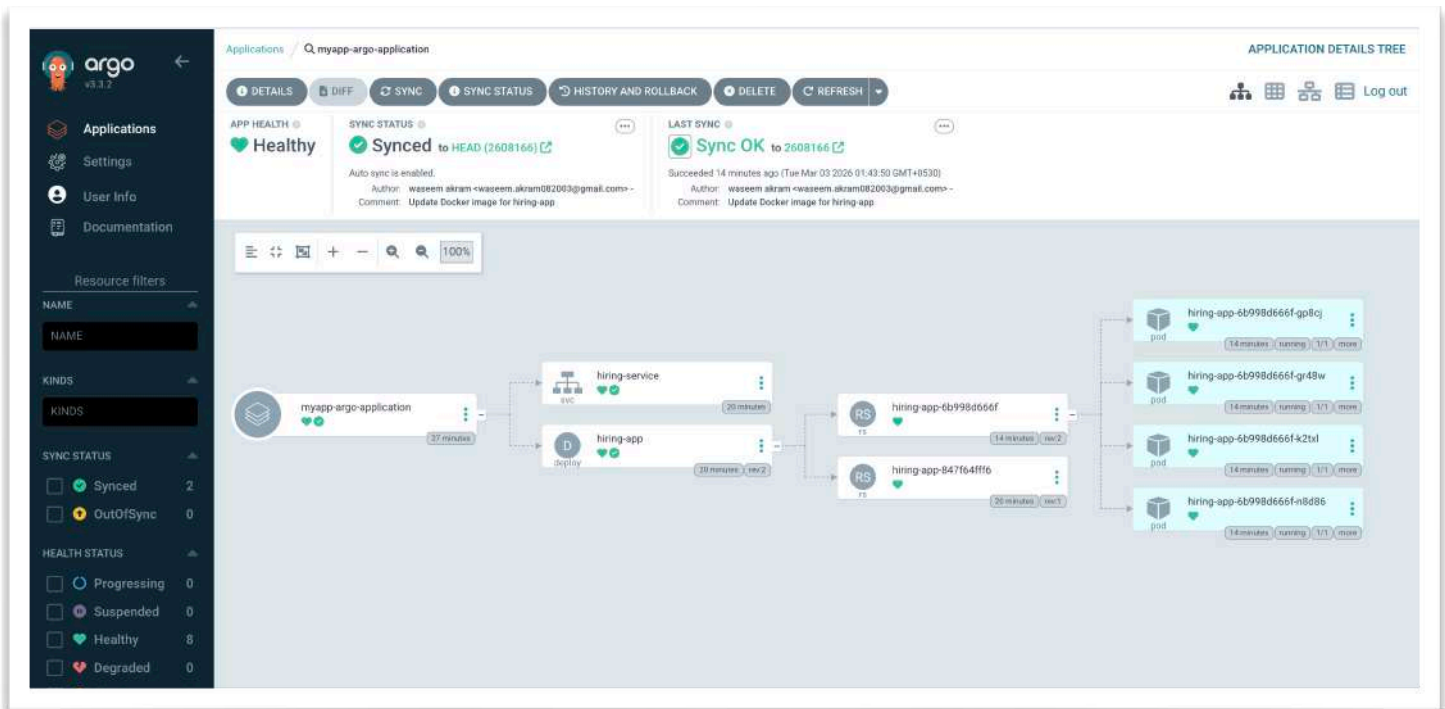
Application should load.





—> now see  
ArgoCD Dashboard.





## GitOps Testing: Changing Replicas from 4 → 6

### initial State (Before Change)

- GitHub `deployment.yaml` contains:

```
replicas: 4
```

- Kubernetes cluster has 4 running pods.
- ArgoCD Application status = **Synced**.
- Git and cluster state match.

This is called:

**Desired State = Live State**

# You Modify GitHub

You edit in GitHub:

```
replicas: 4
```

Change to:

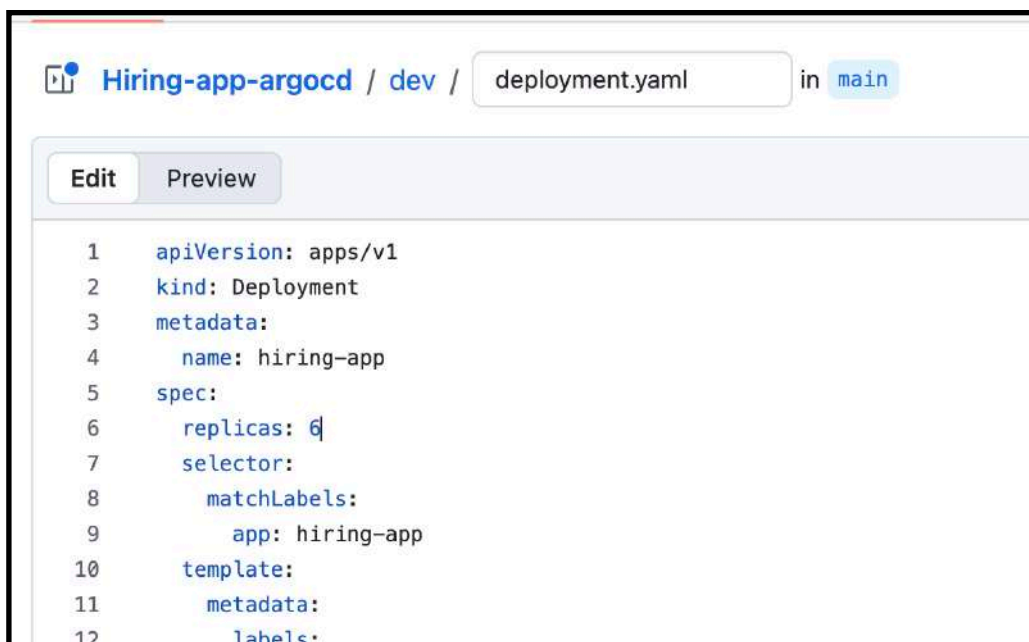
```
replicas: 6
```

Then commit and push.

Now:

- Git Desired State = 6
- Cluster Live State = 4

There is a difference.





## How ArgoCD Detects the Change

ArgoCD continuously:

- Polls the Git repository
- Compares Git state with cluster state

When it sees:

```
Git: replicas = 6
Cluster: replicas = 4
```

It marks the application:

**OutOfSync**

Because you enabled:

```
syncPolicy:
  automated:
    prune: true
    selfHeal: true
```

ArgoCD automatically starts synchronization.

## Kubernetes Reaction

After new Deployment spec is applied:

- Kubernetes sees replica count changed  $4 \rightarrow 6$ .
- It creates 2 additional pods.
- Rolling update occurs.

You can see this using:

```
kubectl get pods -n myapp -w
```

```

[ubuntu@master:~/Hiring-app-argocd$ kubectl get pods -n myapp -w
NAME                                READY    STATUS    RESTARTS   AGE
hiring-app-6b998d666f-gp8cj        1/1      Running   0           33m
hiring-app-6b998d666f-gr48w        1/1      Running   0           33m
hiring-app-6b998d666f-k2txl        1/1      Running   0           33m
hiring-app-6b998d666f-n8d86        1/1      Running   0           33m
hiring-app-6b998d666f-trp6v        0/1      Pending   0           0s
hiring-app-6b998d666f-xh72v        0/1      Pending   0           0s
hiring-app-6b998d666f-trp6v        0/1      Pending   0           0s
hiring-app-6b998d666f-xh72v        0/1      Pending   0           0s
hiring-app-6b998d666f-trp6v        0/1      ContainerCreating   0           0s
hiring-app-6b998d666f-xh72v        0/1      ContainerCreating   0           0s
hiring-app-6b998d666f-xh72v        0/1      ContainerCreating   0           0s
hiring-app-6b998d666f-trp6v        0/1      ContainerCreating   0           0s
hiring-app-6b998d666f-trp6v        1/1      Running   0           1s
hiring-app-6b998d666f-xh72v        1/1      Running   0           1s

```

## Final State

Now:

- Git Desired State = 6
- Cluster Live State = 6
- ArgoCD shows: **Synced**
- Pods running = 6

System is consistent again.

```

^Cubuntu@master:~/Hiring-app-argocd$ kubectl get pods -n myapp
NAME                                READY    STATUS    RESTARTS   AGE
hiring-app-6b998d666f-gp8cj        1/1      Running   0           35m
hiring-app-6b998d666f-gr48w        1/1      Running   0           35m
hiring-app-6b998d666f-k2txl        1/1      Running   0           35m
hiring-app-6b998d666f-n8d86        1/1      Running   0           35m
hiring-app-6b998d666f-trp6v        1/1      Running   0           2m3s
hiring-app-6b998d666f-xh72v        1/1      Running   0           2m3s
ubuntu@master:~/Hiring-app-argocd$

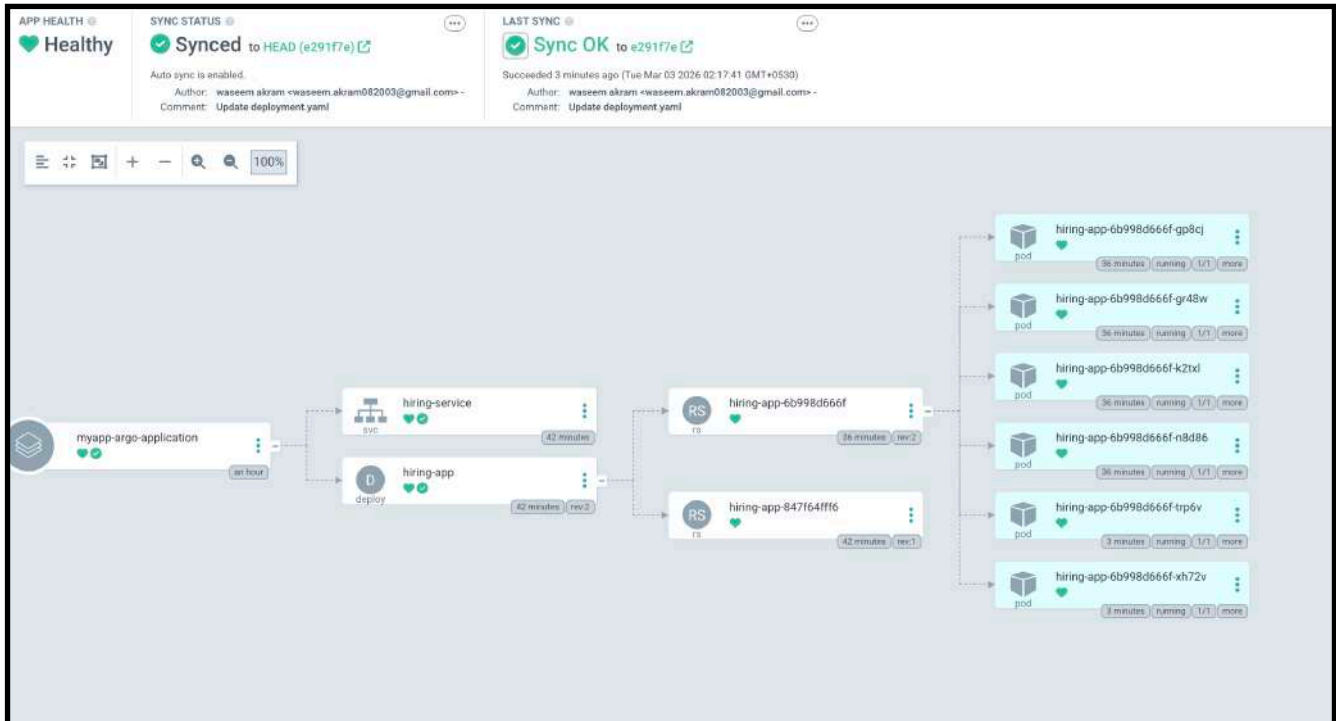
```

```

argocd server pods rs
[ubuntu@master:~$ kubectl get rs -n myapp
NAME                                DESIRED    CURRENT    READY    AGE
hiring-app-6b998d666f              6           6           6        43m

```

—> now check your argocd dashboard we can see 6 pods running



Create ArgoCD job to deploy on EKS cluster.

What you have achieved:

- ArgoCD installed and running in EKS.
- Created ArgoCD Application (application.yaml).
- Connected ArgoCD to your GitHub manifest repo.
- Automated sync enabled.
- Deployment and Service created in cluster.
- 4 replicas initially deployed.
- Tested scaling from GitHub (4 → 6).
- Verified pods scaled automatically.
- Verified service is accessible.
- Demonstrated GitOps reconciliation.

# Key Concepts Demonstrated

## Git Is the Source of Truth

You never changed replicas using:

```
kubectl scale
```

Only Git was modified.

## Continuous Reconciliation

ArgoCD continuously compares:

Desired State (Git)

vs

Live State (Cluster)

## Automated Sync

Because automated sync is enabled:

No manual sync button needed.

## Self-Healing

If someone manually changes cluster to 3 replicas:

ArgoCD will detect mismatch and restore it back to 6.

# Simple Architecture Flow

GitHub (replicas=6)

↓

ArgoCD detects change

↓

ArgoCD applies manifest

↓

Kubernetes scales pods

↓

Cluster matches Git

# One-Line Explanation

“ArgoCD continuously monitors Git as the single source of truth. When the replica count was changed from 4 to 6 in GitHub, ArgoCD automatically reconciled the cluster state by updating the Kubernetes Deployment, resulting in two additional pods being created.”



